

Security Review — Salesforce Code Analyzer False-Positive Responses

Managed package **Pledgivo** (namespace `pledgivo`) · responses prepared 2026-10-02 against source commit `e51c2601`

1. Scan

Scanner	Salesforce Code Analyzer 0.53.0 (<code>sf code-analyzer run</code>)
Engines	retire-js 0.39.0, regex 0.40.0, eslint 0.47.0, pmd 0.46.0
Rule selectors	AppExchange + Recommended:Security — the security-review command
Configuration	<code>code-analyzer.yml</code> at the repository root (custom PMD rulesets and the ESLint options file it names; no rule is disabled)
Target	<code>base/</code> — the whole packaged source
Findings reported	23 across 3 rules, every one answered below
Source commit	<code>e51c2601</code>

2. How to read this document

Code Analyzer reports one finding per rule hit, at a single file and line. Section 4 answers each finding once, with a response ID (**R-*nnn***), grouped by rule. Each response states the scanner's claim in the scanner's own words, then why that claim is false at this site, the supporting detail, the evidence a reviewer can check against the code, and the sources the response relies on. The verdicts are the same ones CI enforces: the project keeps them in `scripts/lint/analyzer-exceptions.json`, a build step fails on any High or security-tagged finding with no verdict and on any verdict whose finding no longer exists, and the docs site publishes the same record. Appendix A lists every finding individually with its message and response ID.

Recurring facts the responses rely on:

- **Secrets live in Named Credentials only.** No Stripe key, token or signing secret exists anywhere in package source or in a custom field. The fields that hold a Stripe *credential* hold the API name of a Named Credential the subscriber administrator builds in Setup, consumed as `'callout:' + name`; Apex cannot read the credential's contents back.
- **Capability tokens are hashed or single-purpose.** The portal access code is stored only as a hash; the receipt and staging tokens are 256-bit random values (64 hex characters) that address one record each and are useless outside the flow that issued them.
- **The FFLib trigger layer's CRUD re-check is redundant with the platform's.** Where the package disables it, the DML still runs under the platform's own object and field permissions; the switch only stops a second, coarser check from blocking a rollout the user is entitled to trigger.
- **Stripe.js must be loaded from Stripe.** Stripe's PCI guidance requires the script to be served from `js.stripe.com` and never bundled or version-pinned; it runs in a Visualforce page inside an iframe that carries no Salesforce session.

3. Summary by rule

Rule	Engine	Severity	Findings	False positive	Working as designed	Third-party
ApexDangerousMethods	pmd	Moderate	3	0	3	0
ProtectSensitiveData	pmd	Moderate	18	10	8	0
LoadJavaScriptHtmlScript	pmd	High	2	0	2	0
Total			23	10	13	0

4. Responses

ApexDangerousMethods — 3 findings (pmd, Moderate; Recommended, Security, Apex)

Reported finding	Location	Response
R-001 ApexDangerousMethods Working as designed	classes/donation/domain/DonationDesignations.cls:36 Calling potentially dangerous method	SCANNER'S CLAIM <i>Calling potentially dangerous method</i> — reported by ApexDangerousMethods (pmd, Moderate) at classes/donation/domain/DonationDesignations.cls:36 WHY THE CLAIM IS FALSE The claim that this call "opens the door to several attacks" by disabling CRUD security is false. The call switches off only FFLib's own trigger-layer assertion, fflib_SObjectDomain.Configuration.EnforcingTriggerCRUDSecurity, a library check that the vendored base class layers above the platform. It changes nothing about how the DML executes: every write that fires this domain still runs under Salesforce's own object permissions, field-level security and sharing, which no Apex call can disable. This domain only recomputes Designation__c fund totals when a Donation_Designation__c row changes. That object is a master-detail detail of the standard Opportunity, so a second-generation package cannot ship an object permission on it and isCreateable() is false for every user without Modify All Data; with the library check left on, FFLib throws Permission to create an Donation_Designation__c denied. on a rollup the user is entitled to trigger, and the fund total silently stops updating. The constructor is the only site and carries its reason in the // doc: comment above it. SUPPORTING DETAIL Configuration.disableTriggerCRUDSecurity() in a rollup domain constructor. The FFLib domain layer re-asserts isCreateable() on every record even when the DML is running in SYSTEM_MODE, which makes an internally-triggered rollup fail for any user who cannot directly create the rolled-up record. These two domains recompute totals only— Designation__c and recurring-donation aggregates—and the call scopes the opt-out to the FFLib trigger check alone. The DML itself still runs under the platform's rules; the platform's own CRUD, FLS and sharing enforcement is untouched. Without it, a non-admin donation write silently stops rolling up fund totals. • At this site: Designation total rollup. Without the opt-out the rollup silently stops recomputing fund totals for non-admin users. • What the rule checks: PMD's ApexDangerousMethods flags Configuration.disableTriggerCRUDSecurity(), which turns off FFLib's own CRUD assertion inside the trigger layer. EVIDENCE Both call sites are annotated in place. In RecurringDonations the call sits behind an explicit suppressTriggerCrudCheck flag rather than being unconditional. The wider reasoning—that field permissions are inert without object access on a master-detail detail object—is written up in the Change Management section of AGENTS.md and in the EventTicketTypesSelector class header. SOURCES

Reported finding	Location	Response
		• https://docs.pmd-code.org/pmd-doc-7.26.0/pmd_rules_apex_security.html#apexdangerousmethods — "Checks against calling dangerous methods"; on <code>Configuration.disableTriggerCRUDSecurity()</code>: "Disabling CRUD security opens the door to several attacks and requires manual validation, which is unreliable." • https://github.com/apex-enterprise-patterns/fflib-apex-common/blob/master/sfdx-source/apex-common/main/classes/fflib_SObjectDomain.cls — <code>Configuration.EnforcingTriggerCRUDSecurity</code> is documented as "True if the base class is checking the users CRUD requirements before invoking trigger methods"; <code>handleAfterInsert()</code> throws <code>DomainException('Permission to create an ' + name + ' denied.')</code> when it is on and <code>SObjectDescribe.isCreateable()</code> is false. It is a library check, not a platform setting. • AGENTS.md, "Change Management — Locked Decisions Protocol": why a detail of a standard object cannot be granted by a 2GP permission set, so its field permissions are inert and <code>isCreateable()</code> is false for every non-admin user.

Reported finding	Location	Response
R-002 ApexDangerousMethods Working as designed	classes/donor/domain/Contacts.cls:52 Calling potentially dangerous method	SCANNER'S CLAIM <i>Calling potentially dangerous method</i> — reported by ApexDangerousMethods (pmd, Moderate) at classes/donor/domain/Contacts.cls:52 WHY THE CLAIM IS FALSE The claim is false for the same reason: The call switches off only FFLib's own trigger-layer assertion, <code>fflib_SObjectDomain.Configuration.EnforcingTriggerCRUDSecurity</code>, a library check that the vendored base class layers above the platform. It changes nothing about how the DML executes: every write that fires this domain still runs under Salesforce's own object permissions, field-level security and sharing, which no Apex call can disable. The call is guarded by <code>suppressTriggerCrudCheck</code>, which only <code>CampaignService</code>'s donor-rollup commit sets, saving the prior value and restoring it in a <code>finally</code>. That commit writes three rollup fields on the donor Contact (<code>Total_Donated__c</code>, <code>Gift_Count__c</code>, <code>Last_Donation_Date__c</code>) after a gift is finalized, in the transaction of whoever saved the gift, including the site guest user, who cannot hold Edit on Contact. The rollup values are computed server-side from the gift that was just committed; no caller-supplied value reaches the Contact, and the platform's own permission checks on the write are untouched. SUPPORTING DETAIL Configuration.disableTriggerCRUDSecurity() in a rollup domain constructor. The FFLib domain layer re-asserts <code>isCreateable()</code> on every record even when the DML is running in <code>SYSTEM_MODE</code>, which makes an internally-triggered rollup fail for any user who cannot directly create the rolled-up record. These two domains recompute totals only—<code>Designation__c</code> and recurring-donation aggregates—and the call scopes the opt-out to the FFLib trigger check alone. The DML itself still runs under the platform's rules; the platform's own CRUD, FLS and sharing enforcement is untouched. Without it, a non-admin donation write silently stops rolling up fund totals. • At this site: Donor rollup (<code>Total_Donated__c</code>, <code>Gift_Count__c</code>, <code>Last_Donation_Date__c</code>), behind <code>suppressTriggerCrudCheck</code>, which only <code>CampaignService</code>'s rollup commit sets and restores in <code>finally</code>. Without it a staff user who can edit a donation but not a Contact cannot close a gift. • What the rule checks: PMD's ApexDangerousMethods flags <code>Configuration.disableTriggerCRUDSecurity()</code>, which turns off FFLib's own CRUD assertion inside the trigger layer. EVIDENCE Both call sites are annotated in place. In <code>RecurringDonations</code> the call sits behind an explicit <code>suppressTriggerCrudCheck</code> flag rather than being unconditional. The wider reasoning—that field permissions are inert without object access on a master-detail detail object—is written up in the Change Management section of AGENTS.md and in the <code>EventTicketTypesSelector</code> class header. SOURCES

Reported finding	Location	Response
		• https://docs.pmd-code.org/pmd-doc-7.26.0/pmd_rules_apex_security.html#apexdangerousmethods — "Checks against calling dangerous methods"; on <code>Configuration.disableTriggerCRUDSecurity()</code>: "Disabling CRUD security opens the door to several attacks and requires manual validation, which is unreliable." • https://github.com/apex-enterprise-patterns/fflib-apex-common/blob/master/sfdx-source/apex-common/main/classes/fflib_SObjectDomain.cls — <code>Configuration.EnforcingTriggerCRUDSecurity</code> is documented as "True if the base class is checking the users CRUD requirements before invoking trigger methods"; <code>handleAfterInsert()</code> throws <code>DomainException('Permission to create an ' + name + ' denied.')</code> when it is on and <code>SObjectDescribe.isCreateable()</code> is false. It is a library check, not a platform setting. • AGENTS.md, "Change Management — Locked Decisions Protocol": why a detail of a standard object cannot be granted by a 2GP permission set, so its field permissions are inert and <code>isCreateable()</code> is false for every non-admin user.

Reported finding	Location	Response
R-003 ApexDangerousMethods Working as designed	classes/recurring/domain/RecurringDonations.cls:61 Calling potentially dangerous method	SCANNER'S CLAIM <i>Calling potentially dangerous method</i> — reported by ApexDangerousMethods (pmd, Moderate) at classes/recurring/domain/RecurringDonations.cls:61 WHY THE CLAIM IS FALSE The claim is false for the same reason: The call switches off only FFLib's own trigger-layer assertion, <code>fflib_SObjectDomain.Configuration.EnforcingTriggerCRUDSecurity</code>, a library check that the vendored base class layers above the platform. It changes nothing about how the DML executes: every write that fires this domain still runs under Salesforce's own object permissions, field-level security and sharing, which no Apex call can disable. Here the call is not even unconditional. It runs only while the static <code>suppressTriggerCrudCheck</code> flag is true, and the sole code that sets the flag is the donor-portal path in <code>DonorPortalService</code> that updates a schedule on the donor's own behalf, which sets it immediately before the commit and clears it in the next statement. In every other transaction the FFLib assertion stays on. The donor is authorized by a verified access code before that path is reached, and the guest user it runs as cannot hold Edit on the object at all (a platform limit of the guest licence), which is exactly why the library check would otherwise block a legitimate update. SUPPORTING DETAIL Configuration.disableTriggerCRUDSecurity() in a rollout domain constructor. The FFLib domain layer re-asserts <code>isCreateable()</code> on every record even when the DML is running in <code>SYSTEM_MODE</code>, which makes an internally-triggered rollout fail for any user who cannot directly create the rolled-up record. These two domains recompute totals only—<code>Designation__c</code> and recurring-donation aggregates—and the call scopes the opt-out to the FFLib trigger check alone. The DML itself still runs under the platform's rules; the platform's own CRUD, FLS and sharing enforcement is untouched. Without it, a non-admin donation write silently stops rolling up fund totals. • At this site: Same call, behind the explicit <code>suppressTriggerCrudCheck</code> flag rather than unconditional. • What the rule checks: PMD's ApexDangerousMethods flags <code>Configuration.disableTriggerCRUDSecurity()</code>, which turns off FFLib's own CRUD assertion inside the trigger layer. EVIDENCE Both call sites are annotated in place. In <code>RecurringDonations</code> the call sits behind an explicit <code>suppressTriggerCrudCheck</code> flag rather than being unconditional. The wider reasoning—that field permissions are inert without object access on a master-detail detail object—is written up in the Change Management section of AGENTS.md and in the <code>EventTicketTypesSelector</code> class header. SOURCES • https://docs.pmd-code.org/pmd-doc-7.26.0/pmd_rules_apex_security.html#apexdangerousmethods — "Checks against calling dangerous methods"; on <code>Configuration.disableTriggerCRUDSecurity()</code>: "Disabling CRUD security opens the door to several attacks and requires manual validation, which is unreliable."

Reported finding	Location	Response
		• https://github.com/apex-enterprise-patterns/fflib-apex-common/blob/master/sfdx-source/apex-common/main/classes/fflib_SObjectDomain.cls — Configuration.EnforcingTriggerCRUDSecurity is documented as "True if the base class is checking the users CRUD requirements before invoking trigger methods"; handleAfterInsert() throws DomainException('Permission to create an ' + name + ' denied.') when it is on and SObjectDescribe.isCreateable() is false. It is a library check, not a platform setting. • AGENTS.md, "Change Management — Locked Decisions Protocol": why a detail of a standard object cannot be granted by a 2GP permission set, so its field permissions are inert and isCreateable() is false for every non-admin user.

ProtectSensitiveData — 18 findings (pmd, Moderate; AppExchange, Security, XML)

Reported finding	Location	Response
R-004 ProtectSensitiveData False positive	objects/Campaign_Design__c/fields/Theme_Tokens_JSON__c.field-meta.xml:1 Theme_Tokens_JSON__c is a potential auth token in a custom object: Campaign_Design__c	SCANNER'S CLAIM <i>Theme_Tokens_JSON__c is a potential auth token in a custom object: Campaign_Design__c</i> — reported by ProtectSensitiveData (pmd, Moderate) at objects/Campaign_Design__c/fields/Theme_Tokens_JSON__c.field-meta.xml:1 WHY THE CLAIM IS FALSE The claim that this field "is a potential auth token" is false: the rule matched on the substring <i>*token*</i>, and the field is a Long Text Area holding a JSON map of CSS custom-property overrides (design tokens such as a button colour or a heading font) for a fundraising page design. The service layer filters every key against a packaged catalog (<i>Theme_Token__mdt</i>) and rejects unsafe values before a page reads it. It is admin-edited presentation data on a per-design record; it authenticates nothing and could not be stored in a protected setting. SUPPORTING DETAIL Field names that merely contain sign, auth, token or credit. These fields match on a substring of an ordinary English word and carry nothing sensitive. <i>Designation__c</i> and <i>Default_Designation__c</i> (the fund a gift is earmarked for) and <i>Campaign_Design__c.Theme_Tokens_JSON__c</i> match on <i>*sign*</i> and <i>*token*</i> — one is a lookup to a program, the other is a JSON map of CSS design-token overrides that the service layer filters against a packaged catalog. <i>Email_Signature__c</i> is the sign-off line printed on outbound email. <i>Re_Auth_Required__c</i> and <i>Re_Auth_Link_Sent_At__c</i> match on <i>*auth*</i> and are a checkbox and a timestamp recording that a donor must complete 3-D Secure on-session for a renewal and when they were last reminded. <i>Portal_Token_TTL_Hours__c</i> is a number of hours. <i>Soft_Credit_Automation_Enabled__c</i> matches on <i>*credit*</i> and is a feature toggle for soft-credit rollups; it has nothing to do with a payment card. A Protected Custom Setting could not hold any of them: they are per-record business data, and the two on <i>Settings__c</i> are admin-visible configuration that the Settings console edits. • At this site: CSS design-token overrides as JSON; <i>*token*</i> means a CSS custom property. • What the rule checks: PMD's ProtectSensitiveData (AppExchange rule set) flags a custom field whose API name matches a list of sensitive-looking substrings, and asks that such data be stored in Protected Custom Settings or Protected Custom Metadata instead. EVIDENCE Each field's <code><description></code> in <code>base/main/default/objects/</code> says what it holds. The package stores no secret in any Custom Setting, protected or not — the org-wide credentials live only in Named Credentials (FP-4 in <code>docs/security/appexchange-security-review-readiness.md</code>), which is what the rule's remediation is for. SOURCES • Code Analyzer's own rule definition (<code>AppExchange.xml.xml</code> inside the bundled <code>sfca-pmd-rules</code> jar, class <code>com.salesforce.security.pmd.xml.DetectSecretsInCustomObjects</code>): the rule message is "To store se

Reported finding	Location	Response
		crets, use Protected Custom settings or Protected Custom metadata." It is keyed on the field's API name, which is why an ordinary word that contains *sign*, *auth*, *token* or *credit* is reported with the message "is a potential auth token". • https://trailhead.salesforce.com/content/learn/modules/secure-secrets-storage/protect-secrets-using-platform-features — the Trailhead module the rule description refers to; it defines the remediation as storing "API keys or other secret keys" in protected custom metadata or protected custom settings. None of these fields holds a key. • https://developer.salesforce.com/docs/platform/salesforce-code-analyzer/guide/rules-pmd-appexchange.html#protectsensitivedata — the scanner's reference for this rule (page could not be retrieved while this document was prepared: the host answers automated requests with HTTP 403, so it is cited as the scanner's own reference link and not quoted). • base/main/default/objects/<Object>/fields/<Field>.field-meta.xml — each flagged field's <description> and <inlineHelpText> state what it holds.

Reported finding	Location	Response
R-005 ProtectSensitiveData False positive	objects/Donation_Designation__c/fields/Designation__c.field-meta.xml:1 Designation__c is a potential auth token in a custom object: Donation_Designation__c	SCANNER'S CLAIM <i>Designation__c is a potential auth token in a custom object: Donation_Designation__c</i> — reported by ProtectSensitiveData (pmd, Moderate) at objects/Donation_Designation__c/fields/Designation__c.field-meta.xml:1 WHY THE CLAIM IS FALSE The claim that this field "is a potential auth token" is false: the rule matched on the substring <i>*sign*</i> inside <i>*Designation*</i>, the fundraising term for the fund or program a gift is earmarked to. The field is a lookup to a Designation__c record on the row that records one split of a gift. A relationship field holds a record Id, not a secret. SUPPORTING DETAIL Field names that merely contain sign, auth, token or credit. These fields match on a substring of an ordinary English word and carry nothing sensitive. Designation__c and Default_Designation__c (the fund a gift is earmarked for) and Campaign_Design__c.Theme_Tokens_JSON__c match on <i>*sign*</i> and <i>*token*</i> — one is a lookup to a program, the other is a JSON map of CSS design-token overrides that the service layer filters against a packaged catalog. Email_Signature__c is the sign-off line printed on outbound email. Re_Auth_Required__c and Re_Auth_Link_Sent_At__c match on <i>*auth*</i> and are a checkbox and a timestamp recording that a donor must complete 3-D Secure on-session for a renewal and when they were last reminded. Portal_Token_TTL_Hours__c is a number of hours. Soft_Credit_Automation_Enabled__c matches on <i>*credit*</i> and is a feature toggle for soft-credit rollups; it has nothing to do with a payment card. A Protected Custom Setting could not hold any of them: they are per-record business data, and the two on Settings__c are admin-visible configuration that the Settings console edits. • At this site: Lookup to the fund a gift split supports; matches on <i>*sign*</i>. • What the rule checks: PMD's ProtectSensitiveData (AppExchange rule set) flags a custom field whose API name matches a list of sensitive-looking substrings, and asks that such data be stored in Protected Custom Settings or Protected Custom Metadata instead. EVIDENCE Each field's <description> in base/main/default/objects/ says what it holds. The package stores no secret in any Custom Setting, protected or not — the org-wide credentials live only in Named Credentials (FP-4 in docs/security/appexchange-security-review-readiness.md), which is what the rule's remediation is for. SOURCES • Code Analyzer's own rule definition (AppExchange_xml.xml inside the bundled sfca-pmd-rules jar, class com.salesforce.security.pmd.xml.DetectSecretsInCustomObjects): the rule message is "To store secrets, use Protected Custom settings or Protected Custom metadata." It is keyed on the field's API name, which is

Reported finding	Location	Response
		why an ordinary word that contains *sign*, *auth*, *token* or *credit* is reported with the message "is a potential authentication token". • https://trailhead.salesforce.com/content/learn/modules/secure-secrets-storage/protect-secrets-using-platform-features — the Trailhead module the rule description refers to; it defines the remediation as storing "API keys or other secret keys" in protected custom metadata or protected custom settings. None of these fields holds a key. • https://developer.salesforce.com/docs/platform/salesforce-code-analyzer/guide/rules-pmd-appexchange.html#protectsensitivedata — the scanner's reference for this rule (page could not be retrieved while this document was prepared: the host answers automated requests with HTTP 403, so it is cited as the scanner's own reference link and not quoted). • base/main/default/objects/<Object>/fields/<Field>.field-meta.xml — each flagged field's <code><description></code> and <code><inlineHelpText></code> state what it holds.

Reported finding	Location	Response
R-006 ProtectSensitiveData Working as designed	objects/Donation_Staging__c/fields/Access_Token__c.field-meta.xml:1 Access_Token__c is a potential auth token in a custom object: Donation_Staging__c	SCANNER'S CLAIM <i>Access_Token__c is a potential auth token in a custom object: Donation_Staging__c</i> — reported by ProtectSensitiveData (pmd, Moderate) at objects/Donation_Staging__c/fields/Access_Token__c.field-meta.xml:1 WHY THE CLAIM IS FALSE The claim that this value should be stored in a protected custom setting is false because it is not an org-wide secret and no setting could hold it: it is a per-record capability token minted server-side for one guest checkout. It is a 256-bit random value (SecureTokens.opaque(), 64 hex characters) written to a unique external-Id field on the staging row, scopes three guest actions to that single row, cannot address any other record, and dies with the row when the checkout completes or the staging retention job purges it. Its entropy, expiry, single-use, revocation and guest-FLS properties are written in the field's <description> and tabulated in FP-16 of the readiness document. Holding it in a protected setting would make it org-wide, which is the opposite of the control it provides. SUPPORTING DETAIL Per-record capability tokens, idempotency keys and SHA-256 hashes. None of these is an org-wide secret, and none could live in a setting: each is per-record transactional state. Donation_Staging__c.Access_Token__c is a 256-bit random capability token minted server-side for one guest checkout; it scopes three guest actions to that single staging row, cannot read any other record, and dies with the row. Idempotency_Key__c on Donation_Staging__c and Guest_Request__c is a client-generated request identifier whose only job is to make a double-submit reuse the first attempt; possessing one grants nothing, and the unique index on it is the race backstop. Portal_Access_Token__c.Code_Hash__c and Session_Hash__c hold the SHA-256 of the emailed sign-in code and of the browser session token respectively — the raw value is never stored, the code hash is nulled the moment the code is redeemed, and the session hash is a unique external Id so a per-request lookup is selective. Storing hashes rather than values is precisely the control the rule is asking for. • At this site: 256-bit random capability token scoped to one staging row; properties in FP-16. • What the rule checks: PMD's ProtectSensitiveData (AppExchange rule set) flags a custom field whose API name contains *token*, *key* or *hash*, expecting a secret that belongs in a Protected Custom Setting or Protected Custom Metadata. EVIDENCE The five properties of each URL-borne token (entropy, expiry, single-use, revocation, guest FLS posture) are written in the field <description> and tabulated in FP-16 of docs/security/appexchange-security-review-readiness.md. SecureTokens.opaque() mints the staging token; DonorPortalService writes only hashes to Portal_Access_Token__c. SOURCES

Reported finding	Location	Response
		- Code Analyzer's own rule definition (AppExchange_xml.xml inside the bundled sfca-pmd-rules jar): the rule message is "To store secrets, use Protected Custom settings or Protected Custom metadata." A protected setting is org-wide configuration; these fields are per-record transaction state, so the remediation cannot apply. https://trailhead.salesforce.com/content/learn/modules/secure-secrets-storage/protect-secrets-using-platform-features — protected custom settings and protected custom metadata are the module's stores for "API keys or other secret keys"; a per-checkout nonce or a hash of a one-time code is neither. https://developer.salesforce.com/docs/platform/salesforce-code-analyzer/guide/rules-pmd-appexchange.html#protectsensitivedata — the scanner's reference for this rule (page could not be retrieved while this document was prepared: the host answers automated requests with HTTP 403, so it is cited as the scanner's own reference link and not quoted). - base/main/default/classes/donor/service/DonorPortalService.cls — <code>Code_Hash__c = hashToken(rawCode)</code> on issue, <code>Code_Hash__c = null</code> on redemption ("single-use: replay fails structurally"), <code>Session_Hash__c = hashToken(sessionToken)</code>; the raw values are never written. - docs/security/appexchange-security-review-readiness.md, FP-16 — the token table: entropy, expiry, single-use, revocation and guest-FLS posture for every URL-borne token.

Reported finding	Location	Response
R-007 ProtectSensitiveData Working as designed	objects/Donation_Staging__c/fields/Idempotency_Key__c.field-meta.xml:1 Idempotency_Key__c is a potential auth token in a custom object: Donation_Staging__c	SCANNER'S CLAIM <i>Idempotency_Key__c is a potential auth token in a custom object: Donation_Staging__c</i> — reported by ProtectSensitiveData (pmd, Moderate) at objects/Donation_Staging__c/fields/Idempotency_Key__c.field-meta.xml:1 WHY THE CLAIM IS FALSE The claim is false because the field holds a client-generated identifier for one ticket-checkout attempt, not a credential: its only job is to make a double-submit reuse the first attempt instead of charging twice. Possessing the value grants nothing, it is never used to authorise a read or a write, and the unique external-Id index on it is the race backstop. It is per-record transaction state and cannot live in an org-wide setting. SUPPORTING DETAIL Per-record capability tokens, idempotency keys and SHA-256 hashes. None of these is an org-wide secret, and none could live in a setting: each is per-record transactional state. Donation_Staging__c.Access-Token__c is a 256-bit random capability token minted server-side for one guest checkout; it scopes three guest actions to that single staging row, cannot read any other record, and dies with the row. Idempotency_Key__c on Donation_Staging__c and Guest_Request__c is a client-generated request identifier whose only job is to make a double-submit reuse the first attempt; possessing one grants nothing, and the unique index on it is the race backstop. Portal_Access-Token__c.Code_Hash__c and Session_Hash__c hold the SHA-256 of the emailed sign-in code and of the browser session token respectively — the raw value is never stored, the code hash is nulled the moment the code is redeemed, and the session hash is a unique external Id so a per-request lookup is selective. Storing hashes rather than values is precisely the control the rule is asking for. • At this site: Client-generated ticket-checkout attempt id; unique index is the double-submit backstop. • What the rule checks: PMD's ProtectSensitiveData (AppExchange rule set) flags a custom field whose API name contains *token*, *key* or *hash*, expecting a secret that belongs in a Protected Custom Setting or Protected Custom Metadata. EVIDENCE The five properties of each URL-borne token (entropy, expiry, single-use, revocation, guest FLS posture) are written in the field <description> and tabulated in FP-16 of docs/security/appexchange-security-review-readiness.md. SecureTokens.opaque() mints the staging token; DonorPortalService writes only hashes to Portal_Access-Token__c. SOURCES • Code Analyzer's own rule definition (AppExchange_xml.xml inside the bundled sfca-pmd-rules jar): the rule message is "To store secrets, use Protected Custom settings or Protected Custom metadata." A protected setting is org-wide configuration; these fields are per-record transaction state, so the remediation cannot apply.

Reported finding	Location	Response
		• https://trailhead.salesforce.com/content/learn/modules/secure-secrets-storage/protect-secrets-using-platform-features — protected custom settings and protected custom metadata are the module's stores for "API keys or other secret keys"; a per-checkout nonce or a hash of a one-time code is neither. • https://developer.salesforce.com/docs/platform/salesforce-code-analyzer/guide/rules-pmd-appexchange.html#protectsensitivedata — the scanner's reference for this rule (page could not be retrieved while this document was prepared: the host answers automated requests with HTTP 403, so it is cited as the scanner's own reference link and not quoted). • base/main/default/classes/donor/service/DonorPortalService.cls — <code>Code_Hash__c = hashToken(rawCode)</code> on issue, <code>Code_Hash__c = null</code> on redemption ("single-use: replay fails structurally"), <code>Session_Hash__c = hashToken(sessionToken)</code>; the raw values are never written. • docs/security/appexchange-security-review-readiness.md, FP-16 — the token table: entropy, expiry, single-use, revocation and guest-FLS posture for every URL-borne token.

Reported finding	Location	Response
R-008 ProtectSensitiveData Working as designed	objects/Guest_Request__c/fields/Idempotency_Key__c.field-meta.xml:1 Idempotency_Key__c is a potential auth token in a custom object: Guest_Request__c	SCANNER'S CLAIM <i>Idempotency_Key__c is a potential auth token in a custom object: Guest_Request__c — reported by ProtectSensitiveData (pmd, Moderate) at</i> objects/Guest_Request__c/fields/Idempotency_Key__c.field-meta.xml:1 WHY THE CLAIM IS FALSE The claim is false for the same reason: the field is a client-generated request identifier on a queued donor request (a refund request, an attendee-naming request) so that a double-click cannot queue the same work twice. It authorises nothing; the request itself is authorised by the donor's verified portal access code before the row is created. A per-request nonce cannot be an org-wide protected setting. SUPPORTING DETAIL Per-record capability tokens, idempotency keys and SHA-256 hashes. None of these is an org-wide secret, and none could live in a setting: each is per-record transactional state. Donation_Staging__c.Access-Token__c is a 256-bit random capability token minted server-side for one guest checkout; it scopes three guest actions to that single staging row, cannot read any other record, and dies with the row. Idempotency_Key__c on Donation_Staging__c and Guest_Request__c is a client-generated request identifier whose only job is to make a double-submit reuse the first attempt; possessing one grants nothing, and the unique index on it is the race backstop. Portal_Access-Token__c.Code_Hash__c and Session_Hash__c hold the SHA-256 of the emailed sign-in code and of the browser session token respectively — the raw value is never stored, the code hash is nulled the moment the code is redeemed, and the session hash is a unique external Id so a per-request lookup is selective. Storing hashes rather than values is precisely the control the rule is asking for. • At this site: Client-generated request id so a double-click cannot queue the same work twice. • What the rule checks: PMD's ProtectSensitiveData (AppExchange rule set) flags a custom field whose API name contains *token*, *key* or *hash*, expecting a secret that belongs in a Protected Custom Setting or Protected Custom Metadata. EVIDENCE The five properties of each URL-borne token (entropy, expiry, single-use, revocation, guest FLS posture) are written in the field <code><description></code> and tabulated in FP-16 of docs/security/appexchange-security-review-readiness.md. <code>SecureTokens.opaque()</code> mints the staging token; <code>DonorPortalService</code> writes only hashes to <code>Portal_Access-Token__c</code>. SOURCES • Code Analyzer's own rule definition (AppExchange_xml.xml inside the bundled sfca-pmd-rules jar): the rule message is "To store secrets, use Protected Custom settings or Protected Custom metadata." A protected setting is org-wide configuration; these fields are per-record transaction state, so the remediation cannot apply. • https://trailhead.salesforce.com/content/learn/modules/secure-secrets-storage/protect-secrets-using-platform-features — protected custom settings and protected custom metadata are the module's stores for "API keys or other se

Reported finding	Location	Response
		cret keys"; a per-checkout nonce or a hash of a one-time code is neither. • https://developer.salesforce.com/docs/platform/salesforce-code-analyzer/guide/rules-pmd-appexchange.html#protectsensitivedata — the scanner's reference for this rule (page could not be retrieved while this document was prepared: the host answers automated requests with HTTP 403, so it is cited as the scanner's own reference link and not quoted). • base/main/default/classes/donor/service/DonorPortalService.cls — <code>Code_Hash__c = hashToken(rawCode)</code> on issue, <code>Code_Hash__c = null</code> on redemption ("single-use: replay fails structurally"), <code>Session_Hash__c = hashToken(sessionToken)</code>; the raw values are never written. • docs/security/appexchange-security-review-readiness.md, FP-16 — the token table: entropy, expiry, single-use, revocation and guest-FLS posture for every URL-borne token.

Reported finding	Location	Response
R-009 ProtectSensitiveData Working as designed	objects/Payment_Account__c/fields/Publishable_Key__c.field-meta.xml:1 Publishable_Key__c is a potential auth token in a custom object: Payment_Account__c	SCANNER'S CLAIM <i>Publishable_Key__c is a potential auth token in a custom object: Payment_Account__c</i> — reported by ProtectSensitiveData (pmd, Moderate) at objects/Payment_Account__c/fields/Publishable_Key__c.field-meta.xml:1 WHY THE CLAIM IS FALSE The claim that this field holds a secret that belongs in a protected setting is false: it holds Stripe's publishable key (pk_...), which Stripe documents as safe to expose and designed to be included in front-end code. It can only identify the account and create payment tokens from card details the donor is typing; it cannot create charges, issue refunds or read account data. The package sends it to every donor's browser by construction, because Stripe.js cannot mount the payment form without it. A protected setting exists to hide a value from the subscriber; this value is published on every donation page on purpose. The secret and restricted keys that do authorise charges and refunds are never in any field. SUPPORTING DETAIL Fields that hold a Named Credential's name, not a key. Stripe_Donor_Restricted_Key__c and Stripe_Admin_Restricted_Key__c are Text(80) fields holding the **API name** of a Named Credential the subscriber admin built by hand in Setup, and Apex consumes them as 'callout:' + name. The key itself is inside the Named Credential and is never readable from Apex, a field, or the UI. The fields are per-Payment-Account rather than org-wide because one org can run several gateway accounts (for example one per legal entity), which a hierarchy setting cannot model. Publishable_Key__c holds Stripe's publishable key, which Stripe designs to be embedded in browser-side code and is sent to every donor's browser by construction; it authorises nothing on its own. • At this site: Stripe publishable key — designed to be sent to the browser; secret key is never in a field. • What the rule checks: PMD's ProtectSensitiveData (AppExchange rule set) flags a custom field whose API name contains *key*, expecting the value to be a secret that belongs in a Protected Custom Setting or Protected Custom Metadata. EVIDENCE Both restricted-key fields say in their <description> that they hold a name and that an unrestricted secret key is not an acceptable value for the credential behind it; StripeGateway.cls builds the endpoint as 'callout:' + acct.Stripe_Donor_Restricted_Key__c. AGENTS.md Rule 9 makes this the required pattern, and FP-4 in docs/security/appexchange-security-review-readiness.md documents it for the reviewer. SOURCES • https://docs.stripe.com/keys — the publishable key (pk_...) is "Safe to expose: Yes ... It can identify your account and create tokens or PaymentMethods from payment details, but it can't perform sensitive operations such as creating charges or reading account data. You can include it in front-end code"; restricted and secret keys are not safe to expose and "Don't put keys in source code or configuration files."

Reported finding	Location	Response
		• https://trailhead.salesforce.com/content/learn/modules/secure-secrets-storage/protect-secrets-using-platform-features — Named Credentials are "a mechanism for securely managing authentication values for external providers and services"; the module's remediation for an API key is a Named Credential or a protected setting, which is exactly where the key lives here. • https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_callouts_named_credentials.htm — Apex callouts through Named Credentials (callout:<name>), the construction <code>StripeGateway.cls</code> uses (page could not be retrieved while this document was prepared: the host answers automated requests with HTTP 403, so it is cited as the scanner's own reference link and not quoted). • <code>base/main/default/classes/payment/gateway/StripeGateway.cls</code> — <code>this.donorEndpoint = 'callout:' + acct.Stripe_Donor_Restricted_Key__c;</code> and the matching admin line: the field value is a Named Credential name. • <code>docs/security/appexchange-security-review-readiness.md</code>, FP-4 — the Named-Credentials-only rule and the reviewer-facing write-up.

Reported finding	Location	Response
R-010 ProtectSensitiveData Working as designed	objects/Payment_Account__c/fields/Stripe_Admin_Restricted_Key__c.field-meta.xml:1 Stripe_Admin_Restricted_Key__c is a potential auth token in a custom object: Payment_Account__c	SCANNER'S CLAIM <i>Stripe_Admin_Restricted_Key__c is a potential auth token in a custom object: Payment_Account__c — reported by ProtectSensitiveData (pmd, Moderate) at objects/Payment_Account__c/fields/Stripe_Admin_Restricted_Key__c.field-meta.xml:1</i> WHY THE CLAIM IS FALSE The claim that this field is "a potential auth token" that should move to a protected setting is false: the field is a Text(80) that holds the API name of a Named Credential, not a key. The subscriber administrator builds that Named Credential by hand in Setup and pastes the restricted key into it; the package consumes the field as 'callout:' + name and can never read the key back, which is exactly the Named Credential pattern the rule's remediation guidance describes. The field's <description> states that it holds a name and that an unrestricted secret key is not an acceptable value for the credential behind it. It is per payment account rather than org-wide because one organisation can run several gateway accounts, which a hierarchy setting cannot model. SUPPORTING DETAIL Fields that hold a Named Credential's name, not a key. Stripe_Donor_Restricted_Key__c and Stripe_Admin_Restricted_Key__c are Text(80) fields holding the **API name** of a Named Credential the subscriber admin built by hand in Setup, and Apex consumes them as 'callout:' + name. The key itself is inside the Named Credential and is never readable from Apex, a field, or the UI. The fields are per-Payment-Account rather than org-wide because one org can run several gateway accounts (for example one per legal entity), which a hierarchy setting cannot model. Publishable_Key__c holds Stripe's publishable key, which Stripe designs to be embedded in browser-side code and is sent to every donor's browser by construction; it authorises nothing on its own. • At this site: Text(80): API name of the Named Credential holding the back-office restricted key. • What the rule checks: PMD's ProtectSensitiveData (AppExchange rule set) flags a custom field whose API name contains *key*, expecting the value to be a secret that belongs in a Protected Custom Setting or Protected Custom Metadata. EVIDENCE Both restricted-key fields say in their <description> that they hold a name and that an unrestricted secret key is not an acceptable value for the credential behind it; StripeGateway.cls builds the endpoint as 'callout:' + acct.Stripe_Donor_Restricted_Key__c. AGENTS.md Rule 9 makes this the required pattern, and FP-4 in docs/security/appexchange-security-review-readiness.md documents it for the reviewer. SOURCES • https://docs.stripe.com/keys — the publishable key (pk_...) is "Safe to expose: Yes ... It can identify your account and create tokens or PaymentMethods from payment details, but it can't perform sensitive operations such as creatin

Reported finding	Location	Response
		g charges or reading account data. You can include it in front-end code"; restricted and secret keys are not safe to expose and "Don't put keys in source code or configuration files." • https://trailhead.salesforce.com/content/learn/modules/secure-secrets-storage/protect-secrets-using-platform-features — Named Credentials are "a mechanism for securely managing authentication values for external providers and services"; the module's remediation for an API key is a Named Credential or a protected setting, which is exactly where the key lives here. • https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_callouts_named_credentials.htm — Apex callouts through Named Credentials (callout:<name>), the construction <code>StripeGateway.cls</code> uses (page could not be retrieved while this document was prepared: the host answers automated requests with HTTP 403, so it is cited as the scanner's own reference link and not quoted). • <code>base/main/default/classes/payment/gateway/StripeGateway.cls</code> — <code>this.donorEndpoint = 'callout:' + acct.Stripe_Donor_Restricted_Key__c;</code> and the matching admin line: the field value is a Named Credential name. • <code>docs/security/appexchange-security-review-readiness.md</code>, FP-4 — the Named-Credentials-only rule and the reviewer-facing write-up.

Reported finding	Location	Response
R-011 ProtectSensitiveData Working as designed	objects/Payment_Account__c/fields/Stripe_Donor_Restricted_Key__c.field-meta.xml:1 Stripe_Donor_Restricted_Key__c is a potential auth token in a custom object: Payment_Account__c	SCANNER'S CLAIM <i>Stripe_Donor_Restricted_Key__c is a potential auth token in a custom object: Payment_Account__c — reported by ProtectSensitiveData (pmd, Moderate) at</i> objects/Payment_Account__c/fields/Stripe_Donor_Restricted_Key__c.field-meta.xml:1 WHY THE CLAIM IS FALSE The claim is false for the same reason: the field is a Text(80) holding the API name of the Named Credential that carries the charge-only restricted key, consumed as 'callout:' + acct.Stripe_Donor_Restricted_Key__c in StripeGateway.cls. The key itself lives only inside the Named Credential the administrator built in Setup and is not readable from Apex, from the field or from any UI. Moving the field to a protected setting would move a credential name, not a secret, and would lose the per-account scope. SUPPORTING DETAIL Fields that hold a Named Credential's name, not a key. Stripe_Donor_Restricted_Key__c and Stripe_Admin_Restricted_Key__c are Text(80) fields holding the **API name** of a Named Credential the subscriber admin built by hand in Setup, and Apex consumes them as 'callout:' + name. The key itself is inside the Named Credential and is never readable from Apex, a field, or the UI. The fields are per-Payment-Account rather than org-wide because one org can run several gateway accounts (for example one per legal entity), which a hierarchy setting cannot model. Publishable_Key__c holds Stripe's publishable key, which Stripe designs to be embedded in browser-side code and is sent to every donor's browser by construction; it authorises nothing on its own. • At this site: Text(80): API name of the Named Credential holding the charge-only restricted key. • What the rule checks: PMD's ProtectSensitiveData (AppExchange rule set) flags a custom field whose API name contains *key*, expecting the value to be a secret that belongs in a Protected Custom Setting or Protected Custom Metadata. EVIDENCE Both restricted-key fields say in their <description> that they hold a name and that an unrestricted secret key is not an acceptable value for the credential behind it; StripeGateway.cls builds the endpoint as 'callout:' + acct.Stripe_Donor_Restricted_Key__c. AGENTS.md Rule 9 makes this the required pattern, and FP-4 in docs/security/appexchange-security-review-readiness.md documents it for the reviewer. SOURCES • https://docs.stripe.com/keys — the publishable key (pk_...) is "Safe to expose: Yes ... It can identify your account and create tokens or PaymentMethods from payment details, but it can't perform sensitive operations such as creating charges or reading account data. You can include it in front-end code"; restricted and secret keys are not safe to expose and "Don't put keys in source code or configuration files."

Reported finding	Location	Response
		• https://trailhead.salesforce.com/content/learn/modules/secure-secrets-storage/protect-secrets-using-platform-features — Named Credentials are "a mechanism for securely managing authentication values for external providers and services"; the module's remediation for an API key is a Named Credential or a protected setting, which is exactly where the key lives here. • https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_callouts_named_credentials.htm — Apex callouts through Named Credentials (callout:<name>), the construction <code>StripeGateway.cls</code> uses (page could not be retrieved while this document was prepared: the host answers automated requests with HTTP 403, so it is cited as the scanner's own reference link and not quoted). • <code>base/main/default/classes/payment/gateway/StripeGateway.cls</code> — <code>this.donorEndpoint = 'callout:' + acct.Stripe_Donor_Restricted_Key__c;</code> and the matching admin line: the field value is a Named Credential name. • <code>docs/security/appexchange-security-review-readiness.md</code>, FP-4 — the Named-Credentials-only rule and the reviewer-facing write-up.

Reported finding	Location	Response
R-012 ProtectSensitiveData Working as designed	objects/Portal_Access-Token__c/fields/Code_Hash__c.field-meta.xml:1 Code_Hash__c is a potential auth token in a custom object: Portal_Access-Token__c	SCANNER'S CLAIM <i>Code_Hash__c is a potential auth token in a custom object: Portal_Access-Token__c — reported by ProtectSensitiveData (pmd, Moderate) at</i> objects/Portal_Access-Token__c/fields/Code_Hash__c.field-meta.xml:1 WHY THE CLAIM IS FALSE The claim is false because the field stores the SHA-256 hash of the emailed sign-in code, never the code itself: DonorPortalService writes <code>Code_Hash__c = hashToken(rawCode)</code> when a code is issued and sets <code>Code_Hash__c = null</code> the moment it is redeemed, so a stored value can neither reveal the code nor be replayed. Storing a hash in place of a secret is the control the rule exists to encourage, and a per-donor one-time code could not be an org-wide protected setting. SUPPORTING DETAIL Per-record capability tokens, idempotency keys and SHA-256 hashes. None of these is an org-wide secret, and none could live in a setting: each is per-record transactional state. <code>Donation_Staging__c.Access-Token__c</code> is a 256-bit random capability token minted server-side for one guest checkout; it scopes three guest actions to that single staging row, cannot read any other record, and dies with the row. <code>Idempotency_Key__c</code> on <code>Donation_Staging__c</code> and <code>Guest_Request__c</code> is a client-generated request identifier whose only job is to make a double-submit reuse the first attempt; possessing one grants nothing, and the unique index on it is the race backstop. <code>Portal_Access-Token__c.Code_Hash__c</code> and <code>Session_Hash__c</code> hold the SHA-256 of the emailed sign-in code and of the browser session token respectively — the raw value is never stored, the code hash is nulled the moment the code is redeemed, and the session hash is a unique external Id so a per-request lookup is selective. Storing hashes rather than values is precisely the control the rule is asking for. • At this site: SHA-256 of the emailed sign-in code, nulled on redemption; the code itself is never stored. • What the rule checks: PMD's ProtectSensitiveData (AppExchange rule set) flags a custom field whose API name contains <code>*token*</code>, <code>*key*</code> or <code>*hash*</code>, expecting a secret that belongs in a Protected Custom Setting or Protected Custom Metadata. EVIDENCE The five properties of each URL-borne token (entropy, expiry, single-use, revocation, guest FLS posture) are written in the field <code><description></code> and tabulated in FP-16 of docs/security/appexchange-security-review-readiness.md. <code>SecureTokens.opaque()</code> mints the staging token; <code>DonorPortalService</code> writes only hashes to <code>Portal_Access-Token__c</code>. SOURCES • Code Analyzer's own rule definition (AppExchange_xml.xml inside the bundled sfca-pmd-rules jar): the rule message is "To store secrets, use Protected Custom settings or Protected Custom metadata." A protected setting is org-wide configuration; these fields are per-record transaction state, so the remediation cannot apply.

Reported finding	Location	Response
		• https://trailhead.salesforce.com/content/learn/modules/secure-secrets-storage/protect-secrets-using-platform-features — protected custom settings and protected custom metadata are the module's stores for "API keys or other secret keys"; a per-checkout nonce or a hash of a one-time code is neither. • https://developer.salesforce.com/docs/platform/salesforce-code-analyzer/guide/rules-pmd-appexchange.html#protectsensitivedata — the scanner's reference for this rule (page could not be retrieved while this document was prepared: the host answers automated requests with HTTP 403, so it is cited as the scanner's own reference link and not quoted). • base/main/default/classes/donor/service/DonorPortalService.cls — <code>Code_Hash__c = hashToken(rawCode)</code> on issue, <code>Code_Hash__c = null</code> on redemption ("single-use: replay fails structurally"), <code>Session_Hash__c = hashToken(sessionToken)</code>; the raw values are never written. • docs/security/appexchange-security-review-readiness.md, FP-16 — the token table: entropy, expiry, single-use, revocation and guest-FLS posture for every URL-borne token.

Reported finding	Location	Response
R-013 ProtectSensitiveData Working as designed	objects/Portal_Access-Token__c/fields/Session_Hash__c.field-meta.xml:1 Session_Hash__c is a potential auth token in a custom object: Portal_Access-Token__c	SCANNER'S CLAIM <i>Session_Hash__c is a potential auth token in a custom object: Portal_Access-Token__c — reported by ProtectSensitiveData (pmd, Moderate) at</i> objects/Portal_Access-Token__c/fields/Session_Hash__c.field-meta.xml:1 WHY THE CLAIM IS FALSE The claim is false because the field stores the SHA-256 hash of the donor's browser session token (<code>Session_Hash__c = hashToken(sessionToken)</code>), never the token: the raw value exists only in the donor's browser, a per-request lookup compares hashes (<code>PortalAccessTokensSelector</code> selects on <code>Session_Hash__c = :sessionHash</code>), and the unique external-Id index keeps that lookup selective. The row expires on the configured session TTL and is revoked by deletion. A hash of a per-session value is not a secret and cannot be an org-wide setting. SUPPORTING DETAIL Per-record capability tokens, idempotency keys and SHA-256 hashes. None of these is an org-wide secret, and none could live in a setting: each is per-record transactional state. <code>Donation_Staging__c.Access-Token__c</code> is a 256-bit random capability token minted server-side for one guest checkout; it scopes three guest actions to that single staging row, cannot read any other record, and dies with the row. <code>Idempotency_Key__c</code> on <code>Donation_Staging__c</code> and <code>Guest_Request__c</code> is a client-generated request identifier whose only job is to make a double-submit reuse the first attempt; possessing one grants nothing, and the unique index on it is the race backstop. <code>Portal_Access-Token__c.Code_Hash__c</code> and <code>Session_Hash__c</code> hold the SHA-256 of the emailed sign-in code and of the browser session token respectively — the raw value is never stored, the code hash is nulled the moment the code is redeemed, and the session hash is a unique external Id so a per-request lookup is selective. Storing hashes rather than values is precisely the control the rule is asking for. • At this site: SHA-256 of the browser session token, unique external Id; the token itself is never stored. • What the rule checks: PMD's ProtectSensitiveData (AppExchange rule set) flags a custom field whose API name contains <code>*token*</code>, <code>*key*</code> or <code>*hash*</code>, expecting a secret that belongs in a Protected Custom Setting or Protected Custom Metadata. EVIDENCE The five properties of each URL-borne token (entropy, expiry, single-use, revocation, guest FLS posture) are written in the field <code><description></code> and tabulated in FP-16 of docs/security/appexchange-security-review-readiness.md. <code>SecureTokens.opaque()</code> mints the staging token; <code>DonorPortalService</code> writes only hashes to <code>Portal_Access-Token__c</code>. SOURCES • Code Analyzer's own rule definition (AppExchange_xml.xml inside the bundled sfca-pmd-rules jar): the rule message is "To store secrets, use Protected Custom settings or Protected Custom metadata." A protected setting is org-wide configuration; these fields are per-record transaction state, so the remediation cannot apply.

Reported finding	Location	Response
		• https://trailhead.salesforce.com/content/learn/modules/secure-secrets-storage/protect-secrets-using-platform-features — protected custom settings and protected custom metadata are the module's stores for "API keys or other secret keys"; a per-checkout nonce or a hash of a one-time code is neither. • https://developer.salesforce.com/docs/platform/salesforce-code-analyzer/guide/rules-pmd-appexchange.html#protectsensitivedata — the scanner's reference for this rule (page could not be retrieved while this document was prepared: the host answers automated requests with HTTP 403, so it is cited as the scanner's own reference link and not quoted). • base/main/default/classes/donor/service/DonorPortalService.cls — <code>Code_Hash__c = hashToken(rawCode)</code> on issue, <code>Code_Hash__c = null</code> on redemption ("single-use: replay fails structurally"), <code>Session_Hash__c = hashToken(sessionToken)</code>; the raw values are never written. • docs/security/appexchange-security-review-readiness.md, FP-16 — the token table: entropy, expiry, single-use, revocation and guest-FLS posture for every URL-borne token.

Reported finding	Location	Response
R-014 ProtectSensitiveData False positive	objects/Recurring_Designation__c/fields/Designation__c.field-meta.xml:1 Designation__c is a potential auth token in a custom object: Recurring_Designation__c	SCANNER'S CLAIM <i>Designation__c is a potential auth token in a custom object: Recurring_Designation__c</i> — reported by ProtectSensitiveData (pmd, Moderate) at objects/Recurring_Designation__c/fields/Designation__c.field-meta.xml:1 WHY THE CLAIM IS FALSE The claim that this field "is a potential auth token" is false: the rule matched on <i>*sign*</i> inside <i>*Designation*</i>. The field is a lookup to the fund a recurring gift split supports, on the junction object between a recurring schedule and its funds. It is a record Id. SUPPORTING DETAIL Field names that merely contain sign, auth, token or credit. These fields match on a substring of an ordinary English word and carry nothing sensitive. <i>Designation__c</i> and <i>Default_Designation__c</i> (the fund a gift is earmarked for) and <i>Campaign_Design__c.Theme_Tokens_JSON__c</i> match on <i>*sign*</i> and <i>*token*</i> — one is a lookup to a program, the other is a JSON map of CSS design-token overrides that the service layer filters against a packaged catalog. <i>Email_Signature__c</i> is the sign-off line printed on outbound email. <i>Re_Auth_Required__c</i> and <i>Re_Auth_Link_Sent_At__c</i> match on <i>*auth*</i> and are a checkbox and a timestamp recording that a donor must complete 3-D Secure on-session for a renewal and when they were last reminded. <i>Portal_Token_TTL_Hours__c</i> is a number of hours. <i>Soft_Credit_Automation_Enabled__c</i> matches on <i>*credit*</i> and is a feature toggle for soft-credit rollups; it has nothing to do with a payment card. A Protected Custom Setting could not hold any of them: they are per-record business data, and the two on <i>Settings__c</i> are admin-visible configuration that the Settings console edits. • At this site: Lookup to the fund a recurring split supports; matches on <i>*sign*</i>. • What the rule checks: PMD's ProtectSensitiveData (AppExchange rule set) flags a custom field whose API name matches a list of sensitive-looking substrings, and asks that such data be stored in Protected Custom Settings or Protected Custom Metadata instead. EVIDENCE Each field's <code><description></code> in <code>base/main/default/objects/</code> says what it holds. The package stores no secret in any Custom Setting, protected or not — the org-wide credentials live only in Named Credentials (FP-4 in <code>docs/security/appexchange-security-review-readiness.md</code>), which is what the rule's remediation is for. SOURCES • Code Analyzer's own rule definition (<code>AppExchange.xml.xml</code> inside the bundled <code>sfca-pmd-rules</code> jar, class <code>com.salesforce.security.pmd.xml.DetectSecretsInCustomObjects</code>): the rule message is "To store secrets, use Protected Custom settings or Protected Custom metadata." It is keyed on the field's API name, which is why an ordinary word that contains <i>*sign*</i>, <i>*auth*</i>, <i>*token*</i> or <i>*credit*</i> is reported with the message "is a potential auth token".

Reported finding	Location	Response
		• https://trailhead.salesforce.com/content/learn/modules/secure-secrets-storage/protect-secrets-using-platform-features — the Trailhead module the rule description refers to; it defines the remediation as storing "API keys or other secret keys" in protected custom metadata or protected custom settings. None of these fields holds a key. • https://developer.salesforce.com/docs/platform/salesforce-code-analyzer/guide/rules-pmd-appexchange.html#protectsensitivedata — the scanner's reference for this rule (page could not be retrieved while this document was prepared: the host answers automated requests with HTTP 403, so it is cited as the scanner's own reference link and not quoted). • base/main/default/objects/<Object>/fields/<Field>.field-meta.xml — each flagged field's <code><description></code> and <code><inlineHelpText></code> state what it holds.

Reported finding	Location	Response
R-015 ProtectSensitiveData False positive	objects/Recurring_Donation__c/fields/Designation__c.field-meta.xml:1 Designation__c is a potential auth token in a custom object: Recurring_Donation__c	SCANNER'S CLAIM <i>Designation__c is a potential auth token in a custom object: Recurring_Donation__c</i> — reported by ProtectSensitiveData (pmd, Moderate) at objects/Recurring_Donation__c/fields/Designation__c.field-meta.xml:1 WHY THE CLAIM IS FALSE The claim that this field "is a potential auth token" is false: the rule matched on <i>*sign*</i> inside <i>*Designation*</i>. The field is the single-fund lookup on a recurring schedule, used when the schedule is not split across several funds. It is a record Id. SUPPORTING DETAIL Field names that merely contain sign, auth, token or credit. These fields match on a substring of an ordinary English word and carry nothing sensitive. <i>Designation__c</i> and <i>Default_Designation__c</i> (the fund a gift is earmarked for) and <i>Campaign_Design__c.Theme_Tokens_JSON__c</i> match on <i>*sign*</i> and <i>*token*</i> — one is a lookup to a program, the other is a JSON map of CSS design-token overrides that the service layer filters against a packaged catalog. <i>Email_Signature__c</i> is the sign-off line printed on outbound email. <i>Re_Auth_Required__c</i> and <i>Re_Auth_Link_Sent_At__c</i> match on <i>*auth*</i> and are a checkbox and a timestamp recording that a donor must complete 3-D Secure on-session for a renewal and when they were last reminded. <i>Portal_Token_TTL_Hours__c</i> is a number of hours. <i>Soft_Credit_Automation_Enabled__c</i> matches on <i>*credit*</i> and is a feature toggle for soft-credit rollups; it has nothing to do with a payment card. A Protected Custom Setting could not hold any of them: they are per-record business data, and the two on <i>Settings__c</i> are admin-visible configuration that the Settings console edits. • At this site: Single-fund lookup on a recurring schedule; matches on <i>*sign*</i>. • What the rule checks: PMD's ProtectSensitiveData (AppExchange rule set) flags a custom field whose API name matches a list of sensitive-looking substrings, and asks that such data be stored in Protected Custom Settings or Protected Custom Metadata instead. EVIDENCE Each field's <code><description></code> in <code>base/main/default/objects/</code> says what it holds. The package stores no secret in any Custom Setting, protected or not — the org-wide credentials live only in Named Credentials (FP-4 in docs/security/appexchange-security-review-readiness.md), which is what the rule's remediation is for. SOURCES • Code Analyzer's own rule definition (AppExchange_xml.xml inside the bundled sfca-pmd-rules jar, class <code>com.salesforce.security.pmd.xml.DetectSecretsInCustomObjects</code>): the rule message is "To store secrets, use Protected Custom settings or Protected Custom metadata." It is keyed on the field's API name, which is why an ordinary word that contains <i>*sign*</i>, <i>*auth*</i>, <i>*token*</i> or <i>*credit*</i> is reported with the message "is a potential auth token".

Reported finding	Location	Response
		• https://trailhead.salesforce.com/content/learn/modules/secure-secrets-storage/protect-secrets-using-platform-features — the Trailhead module the rule description refers to; it defines the remediation as storing "API keys or other secret keys" in protected custom metadata or protected custom settings. None of these fields holds a key. • https://developer.salesforce.com/docs/platform/salesforce-code-analyzer/guide/rules-pmd-appexchange.html#protectsensitivedata — the scanner's reference for this rule (page could not be retrieved while this document was prepared: the host answers automated requests with HTTP 403, so it is cited as the scanner's own reference link and not quoted). • base/main/default/objects/<Object>/fields/<Field>.field-meta.xml — each flagged field's <code><description></code> and <code><inlineHelpText></code> state what it holds.

Reported finding	Location	Response
R-016 ProtectSensitiveData False positive	objects/Recurring_Donation__c/fields/Re_Auth_Link_Sent_At__c.field-meta.xml:1 Re_Auth_Link_Sent_At__c is a potential auth token in a custom object: Recurring_Donation__c	SCANNER'S CLAIM <i>Re_Auth_Link_Sent_At__c is a potential auth token in a custom object: Recurring_Donation__c</i> — reported by ProtectSensitiveData (pmd, Moderate) at objects/Recurring_Donation__c/fields/Re_Auth_Link_Sent_At__c.field-meta.xml:1 WHY THE CLAIM IS FALSE The claim that this field "is a potential auth token" is false: the rule matched on <i>*auth*</i> inside <i>*Re_Auth*</i>, which here means re-authentication of a card payment, not an API credential. The field is a Date/Time recording when the donor was last emailed a reminder to complete 3-D Secure on-session for a renewal, so the reminder is not sent twice. A timestamp authorises nothing. SUPPORTING DETAIL Field names that merely contain sign, auth, token or credit. These fields match on a substring of an ordinary English word and carry nothing sensitive. <i>Designation__c</i> and <i>Default_Designation__c</i> (the fund a gift is earmarked for) and <i>Campaign_Design__c.Theme_Tokens_JSON__c</i> match on <i>*sign*</i> and <i>*token*</i> — one is a lookup to a program, the other is a JSON map of CSS design-token overrides that the service layer filters against a packaged catalog. <i>Email_Signature__c</i> is the sign-off line printed on outbound email. <i>Re_Auth_Required__c</i> and <i>Re_Auth_Link_Sent_At__c</i> match on <i>*auth*</i> and are a checkbox and a timestamp recording that a donor must complete 3-D Secure on-session for a renewal and when they were last reminded. <i>Portal_Token_TTL_Hours__c</i> is a number of hours. <i>Soft_Credit_Automation_Enabled__c</i> matches on <i>*credit*</i> and is a feature toggle for soft-credit rollups; it has nothing to do with a payment card. A Protected Custom Setting could not hold any of them: they are per-record business data, and the two on <i>Settings__c</i> are admin-visible configuration that the Settings console edits. • At this site: Timestamp of the last 3-D Secure reminder email; matches on <i>*auth*</i>. • What the rule checks: PMD's ProtectSensitiveData (AppExchange rule set) flags a custom field whose API name matches a list of sensitive-looking substrings, and asks that such data be stored in Protected Custom Settings or Protected Custom Metadata instead. EVIDENCE Each field's <code><description></code> in <code>base/main/default/objects/</code> says what it holds. The package stores no secret in any Custom Setting, protected or not — the org-wide credentials live only in Named Credentials (FP-4 in docs/security/appexchange-security-review-readiness.md), which is what the rule's remediation is for. SOURCES • Code Analyzer's own rule definition (AppExchange_xml.xml inside the bundled sfca-pmd-rules jar, class <code>com.salesforce.security.pmd.xml.DetectSecretsInCustomObjects</code>): the rule message is "To store secrets, use Protected Custom settings or Protected Custom metadata." It is keyed on the field's API name, which is

Reported finding	Location	Response
		why an ordinary word that contains *sign*, *auth*, *token* or *credit* is reported with the message "is a potential auth token". • https://trailhead.salesforce.com/content/learn/modules/secure-secrets-storage/protect-secrets-using-platform-features — the Trailhead module the rule description refers to; it defines the remediation as storing "API keys or other secret keys" in protected custom metadata or protected custom settings. None of these fields holds a key. • https://developer.salesforce.com/docs/platform/salesforce-code-analyzer/guide/rules-pmd-appexchange.html#protectsensitivedata — the scanner's reference for this rule (page could not be retrieved while this document was prepared: the host answers automated requests with HTTP 403, so it is cited as the scanner's own reference link and not quoted). • base/main/default/objects/<Object>/fields/<Field>.field-meta.xml — each flagged field's <code><description></code> and <code><inlineHelpText></code> state what it holds.

Reported finding	Location	Response
R-017 ProtectSensitiveData False positive	objects/Recurring_Donation__c/fields/Re_Auth_Required__c.field-meta.xml:1 Re_Auth_Required__c is a potential auth token in a custom object: Recurring_Donation__c	SCANNER'S CLAIM <i>Re_Auth_Required__c is a potential auth token in a custom object: Recurring_Donation__c</i> — reported by ProtectSensitiveData (pmd, Moderate) at objects/Recurring_Donation__c/fields/Re_Auth_Required__c.field-meta.xml:1 WHY THE CLAIM IS FALSE The claim that this field "is a potential auth token" is false: the rule matched on <i>*auth*</i> inside <i>*Re_Auth*</i>. The field is a Checkbox that marks a recurring schedule whose next renewal needs the donor to complete 3-D Secure on-session; it is set from the payment gateway's decline reason and cleared once the donor completes the challenge. A boolean flag is not a secret. SUPPORTING DETAIL Field names that merely contain sign, auth, token or credit. These fields match on a substring of an ordinary English word and carry nothing sensitive. <i>Designation__c</i> and <i>Default_Designation__c</i> (the fund a gift is earmarked for) and <i>Campaign_Design__c.Theme_Tokens_JSON__c</i> match on <i>*sign*</i> and <i>*token*</i> — one is a lookup to a program, the other is a JSON map of CSS design-token overrides that the service layer filters against a packaged catalog. <i>Email_Signature__c</i> is the sign-off line printed on outbound email. <i>Re_Auth_Required__c</i> and <i>Re_Auth_Link_Sent_At__c</i> match on <i>*auth*</i> and are a checkbox and a timestamp recording that a donor must complete 3-D Secure on-session for a renewal and when they were last reminded. <i>Portal_Token_TTL_Hours__c</i> is a number of hours. <i>Soft_Credit_Automation_Enabled__c</i> matches on <i>*credit*</i> and is a feature toggle for soft-credit rollups; it has nothing to do with a payment card. A Protected Custom Setting could not hold any of them: they are per-record business data, and the two on <i>Settings__c</i> are admin-visible configuration that the Settings console edits. • At this site: Checkbox: the donor must complete 3-D Secure on-session; matches on <i>*auth*</i>. • What the rule checks: PMD's ProtectSensitiveData (AppExchange rule set) flags a custom field whose API name matches a list of sensitive-looking substrings, and asks that such data be stored in Protected Custom Settings or Protected Custom Metadata instead. EVIDENCE Each field's <code><description></code> in <code>base/main/default/objects/</code> says what it holds. The package stores no secret in any Custom Setting, protected or not — the org-wide credentials live only in Named Credentials (FP-4 in docs/security/appexchange-security-review-readiness.md), which is what the rule's remediation is for. SOURCES • Code Analyzer's own rule definition (AppExchange_xml.xml inside the bundled sfca-pmd-rules jar, class <code>com.salesforce.security.pmd.xml.DetectSecretsInCustomObjects</code>): the rule message is "To store secrets, use Protected Custom settings or Protected Custom metadata." It is keyed on the field's API name, which is

Reported finding	Location	Response
		why an ordinary word that contains *sign*, *auth*, *token* or *credit* is reported with the message "is a potential auth token". • https://trailhead.salesforce.com/content/learn/modules/secure-secrets-storage/protect-secrets-using-platform-features — the Trailhead module the rule description refers to; it defines the remediation as storing "API keys or other secret keys" in protected custom metadata or protected custom settings. None of these fields holds a key. • https://developer.salesforce.com/docs/platform/salesforce-code-analyzer/guide/rules-pmd-appexchange.html#protectsensitivedata — the scanner's reference for this rule (page could not be retrieved while this document was prepared: the host answers automated requests with HTTP 403, so it is cited as the scanner's own reference link and not quoted). • base/main/default/objects/<Object>/fields/<Field>.field-meta.xml — each flagged field's <code><description></code> and <code><inlineHelpText></code> state what it holds.

Reported finding	Location	Response
R-018 ProtectSensitiveData False positive	objects/Settings__c/fields/Default_Designation__c.field-meta.xml:1 Default_Designation__c is a potential auth token in the object: Settings__c with public visibility	SCANNER'S CLAIM <i>Default_Designation__c is a potential auth token in the object: Settings__c with public visibility — reported by ProtectSensitiveData (pmd, Moderate) at</i> objects/Settings__c/fields/Default_Designation__c.field-meta.xml:1 WHY THE CLAIM IS FALSE The claim that this field "is a potential auth token" is false: the rule matched on <i>*sign*</i> inside <i>*Designation*</i>. The field is an 18-character Text holding the record Id of the fallback fund a gift is credited to when the donor picks none; it is Text only because a hierarchy custom setting cannot declare a lookup. It is edited by an administrator in the Settings console and holds nothing secret. SUPPORTING DETAIL Field names that merely contain sign, auth, token or credit. These fields match on a substring of an ordinary English word and carry nothing sensitive. <i>Designation__c</i> and <i>Default_Designation__c</i> (the fund a gift is earmarked for) and <i>Campaign_Design__c.Theme_Tokens_JSON__c</i> match on <i>*sign*</i> and <i>*token*</i> — one is a lookup to a program, the other is a JSON map of CSS design-token overrides that the service layer filters against a packaged catalog. <i>Email_Signature__c</i> is the sign-off line printed on outbound email. <i>Re_Auth_Required__c</i> and <i>Re_Auth_Link_Sent_At__c</i> match on <i>*auth*</i> and are a checkbox and a timestamp recording that a donor must complete 3-D Secure on-session for a renewal and when they were last reminded. <i>Portal_Token_TTL_Hours__c</i> is a number of hours. <i>Soft_Credit_Automation_Enabled__c</i> matches on <i>*credit*</i> and is a feature toggle for soft-credit rollups; it has nothing to do with a payment card. A Protected Custom Setting could not hold any of them: they are per-record business data, and the two on <i>Settings__c</i> are admin-visible configuration that the Settings console edits. • At this site: 18-character Id of the fallback fund, stored as text because a Custom Setting cannot hold a lookup; matches on <i>*sign*</i>. • What the rule checks: PMD's ProtectSensitiveData (AppExchange rule set) flags a custom field whose API name matches a list of sensitive-looking substrings, and asks that such data be stored in Protected Custom Settings or Protected Custom Metadata instead. EVIDENCE Each field's <code><description></code> in <code>base/main/default/objects/</code> says what it holds. The package stores no secret in any Custom Setting, protected or not — the org-wide credentials live only in Named Credentials (FP-4 in <code>docs/security/appexchange-security-review-readiness.md</code>), which is what the rule's remediation is for. SOURCES • Code Analyzer's own rule definition (<code>AppExchange_xml.xml</code> inside the bundled <code>sfca-pmd-rules</code> jar, class <code>com.salesforce.security.pmd.xml.DetectSecretsInCustomObjects</code>): the rule message is "To store secrets, use Protected Custom settings or Protected Custom metadata." It is keyed on the field's API name, which is

Reported finding	Location	Response
		why an ordinary word that contains *sign*, *auth*, *token* or *credit* is reported with the message "is a potential auth token". • https://trailhead.salesforce.com/content/learn/modules/secure-secrets-storage/protect-secrets-using-platform-features — the Trailhead module the rule description refers to; it defines the remediation as storing "API keys or other secret keys" in protected custom metadata or protected custom settings. None of these fields holds a key. • https://developer.salesforce.com/docs/platform/salesforce-code-analyzer/guide/rules-pmd-appexchange.html#protectsensitivedata — the scanner's reference for this rule (page could not be retrieved while this document was prepared: the host answers automated requests with HTTP 403, so it is cited as the scanner's own reference link and not quoted). • base/main/default/objects/<Object>/fields/<Field>.field-meta.xml — each flagged field's <code><description></code> and <code><inlineHelpText></code> state what it holds.

Reported finding	Location	Response
R-019 ProtectSensitiveData False positive	objects/Settings__c/fields/Email_Signature__c.field-meta.xml:1 Email_Signature__c is a potential auth token in the object: Settings__c with public visibility	SCANNER'S CLAIM <i>Email_Signature__c is a potential auth token in the object: Settings__c with public visibility</i> — reported by ProtectSensitiveData (pmd, Moderate) at objects/Settings__c/fields/Email_Signature__c.field-meta.xml:1 WHY THE CLAIM IS FALSE The claim that this field "is a potential auth token" is false: the rule matched on <i>*sign*</i> inside <i>*Signature*</i>, which here is the sign-off line printed above the footer of every outbound email (for example "The Riverside Foundation team"). It is a short display string an administrator edits, seeded at install from the organisation name, and is sent to every donor by design. SUPPORTING DETAIL Field names that merely contain sign, auth, token or credit. These fields match on a substring of an ordinary English word and carry nothing sensitive. <code>Designation__c</code> and <code>Default_Designation__c</code> (the fund a gift is earmarked for) and <code>Campaign_Design__c.Theme_Tokens_JSON__c</code> match on <i>*sign*</i> and <i>*token*</i> — one is a lookup to a program, the other is a JSON map of CSS design-token overrides that the service layer filters against a packaged catalog. <code>Email_Signature__c</code> is the sign-off line printed on outbound email. <code>Re_Auth_Required__c</code> and <code>Re_Auth_Link_Sent_At__c</code> match on <i>*auth*</i> and are a checkbox and a timestamp recording that a donor must complete 3-D Secure on-session for a renewal and when they were last reminded. <code>Portal_Token_TTL_Hours__c</code> is a number of hours. <code>Soft_Credit_Automation_Enabled__c</code> matches on <i>*credit*</i> and is a feature toggle for soft-credit rollups; it has nothing to do with a payment card. A Protected Custom Setting could not hold any of them: they are per-record business data, and the two on <code>Settings__c</code> are admin-visible configuration that the Settings console edits. • At this site: Sign-off line printed above the footer of every outbound email. • What the rule checks: PMD's ProtectSensitiveData (AppExchange rule set) flags a custom field whose API name matches a list of sensitive-looking substrings, and asks that such data be stored in Protected Custom Settings or Protected Custom Metadata instead. EVIDENCE Each field's <code><description></code> in <code>base/main/default/objects/</code> says what it holds. The package stores no secret in any Custom Setting, protected or not — the org-wide credentials live only in Named Credentials (FP-4 in <code>docs/security/appexchange-security-review-readiness.md</code>), which is what the rule's remediation is for. SOURCES • Code Analyzer's own rule definition (<code>AppExchange_xml.xml</code> inside the bundled <code>sfca-pmd-rules</code> jar, class <code>com.salesforce.security.pmd.xml.DetectSecretsInCustomObjects</code>): the rule message is "To store secrets, use Protected Custom settings or Protected Custom metadata." It is keyed on the field's API name, which is

Reported finding	Location	Response
		why an ordinary word that contains *sign*, *auth*, *token* or *credit* is reported with the message "is a potential auth token". • https://trailhead.salesforce.com/content/learn/modules/secure-secrets-storage/protect-secrets-using-platform-features — the Trailhead module the rule description refers to; it defines the remediation as storing "API keys or other secret keys" in protected custom metadata or protected custom settings. None of these fields holds a key. • https://developer.salesforce.com/docs/platform/salesforce-code-analyzer/guide/rules-pmd-appexchange.html#protectsensitivedata — the scanner's reference for this rule (page could not be retrieved while this document was prepared: the host answers automated requests with HTTP 403, so it is cited as the scanner's own reference link and not quoted). • base/main/default/objects/<Object>/fields/<Field>.field-meta.xml — each flagged field's <code><description></code> and <code><inlineHelpText></code> state what it holds.

Reported finding	Location	Response
R-020 ProtectSensitiveData False positive	objects/Settings__c/fields/Portal_Token_TTL_Hours__c.field-meta.xml:1 Portal_Token_TTL_Hours__c is a potential auth token in the object: Settings__c with public visibility	SCANNER'S CLAIM <i>Portal_Token_TTL_Hours__c is a potential auth token in the object: Settings__c with public visibility — reported by ProtectSensitiveData (pmd, Moderate) at objects/Settings__c/fields/Portal_Token_TTL_Hours__c.field-meta.xml:1</i> WHY THE CLAIM IS FALSE The claim that this field "is a potential auth token" is false: the rule matched on <i>*token*</i>, but the field is a Number: the ceiling, in hours, on how long a donor-portal session may live before the donor must sign in again. It is a configuration value an administrator sets; it contains no token. SUPPORTING DETAIL Field names that merely contain sign, auth, token or credit. These fields match on a substring of an ordinary English word and carry nothing sensitive. <i>Designation__c</i> and <i>Default_Designation__c</i> (the fund a gift is earmarked for) and <i>Campaign_Design__c.Theme_Tokens_JSON__c</i> match on <i>*sign*</i> and <i>*token*</i> — one is a lookup to a program, the other is a JSON map of CSS design-token overrides that the service layer filters against a packaged catalog. <i>Email_Signature__c</i> is the sign-off line printed on outbound email. <i>Re_Auth_Required__c</i> and <i>Re_Auth_Link_Sent_At__c</i> match on <i>*auth*</i> and are a checkbox and a timestamp recording that a donor must complete 3-D Secure on-session for a renewal and when they were last reminded. <i>Portal_Token_TTL_Hours__c</i> is a number of hours. <i>Soft_Credit_Automation_Enabled__c</i> matches on <i>*credit*</i> and is a feature toggle for soft-credit rollups; it has nothing to do with a payment card. A Protected Custom Setting could not hold any of them: they are per-record business data, and the two on <i>Settings__c</i> are admin-visible configuration that the Settings console edits. • At this site: A number of hours — the ceiling on a donor-portal session's lifetime. • What the rule checks: PMD's ProtectSensitiveData (AppExchange rule set) flags a custom field whose API name matches a list of sensitive-looking substrings, and asks that such data be stored in Protected Custom Settings or Protected Custom Metadata instead. EVIDENCE Each field's <code><description></code> in <code>base/main/default/objects/</code> says what it holds. The package stores no secret in any Custom Setting, protected or not — the org-wide credentials live only in Named Credentials (FP-4 in <code>docs/security/appexchange-security-review-readiness.md</code>), which is what the rule's remediation is for. SOURCES • Code Analyzer's own rule definition (<code>AppExchange_xml.xml</code> inside the bundled <code>sfca-pmd-rules</code> jar, class <code>com.salesforce.security.pmd.xml.DetectSecretsInCustomObjects</code>): the rule message is "To store secrets, use Protected Custom settings or Protected Custom metadata." It is keyed on the field's API name, which is why an ordinary word that contains <i>*sign*</i>, <i>*auth*</i>, <i>*token*</i> or <i>*credit*</i> is reported with the message "is a potential auth token".

Reported finding	Location	Response
		• https://trailhead.salesforce.com/content/learn/modules/secure-secrets-storage/protect-secrets-using-platform-features — the Trailhead module the rule description refers to; it defines the remediation as storing "API keys or other secret keys" in protected custom metadata or protected custom settings. None of these fields holds a key. • https://developer.salesforce.com/docs/platform/salesforce-code-analyzer/guide/rules-pmd-appexchange.html#protectsensitivedata — the scanner's reference for this rule (page could not be retrieved while this document was prepared: the host answers automated requests with HTTP 403, so it is cited as the scanner's own reference link and not quoted). • base/main/default/objects/<Object>/fields/<Field>.field-meta.xml — each flagged field's <code><description></code> and <code><inlineHelpText></code> state what it holds.

Reported finding	Location	Response
R-021 ProtectSensitiveData False positive	objects/Settings__c/fields/Soft_Credit_Automation_Enabled__c.field-meta.xml:1 Soft_Credit_Automation_Enabled__c is a potential auth token in the object: Settings__c with public visibility	SCANNER'S CLAIM <i>Soft_Credit_Automation_Enabled__c is a potential auth token in the object: Settings__c with public visibility</i> — reported by ProtectSensitiveData (pmd, Moderate) at objects/Settings__c/fields/Soft_Credit_Automation_Enabled__c.field-meta.xml:1 WHY THE CLAIM IS FALSE The claim that this field "is a potential auth token" is false: the rule matched on <i>*credit*</i>, which here is the fundraising term soft credit (crediting a second person for influencing a gift), not a payment card. The field is a Checkbox that turns the soft-credit rollout on or off. A feature toggle is not a secret. SUPPORTING DETAIL Field names that merely contain sign, auth, token or credit. These fields match on a substring of an ordinary English word and carry nothing sensitive. <i>Designation__c</i> and <i>Default_Designation__c</i> (the fund a gift is earmarked for) and <i>Campaign_Design__c.Theme_Tokens_JSON__c</i> match on <i>*sign*</i> and <i>*token*</i> — one is a lookup to a program, the other is a JSON map of CSS design-token overrides that the service layer filters against a packaged catalog. <i>Email_Signature__c</i> is the sign-off line printed on outbound email. <i>Re_Auth_Required__c</i> and <i>Re_Auth_Link_Sent_At__c</i> match on <i>*auth*</i> and are a checkbox and a timestamp recording that a donor must complete 3-D Secure on-session for a renewal and when they were last reminded. <i>Portal_Token_TTL_Hours__c</i> is a number of hours. <i>Soft_Credit_Automation_Enabled__c</i> matches on <i>*credit*</i> and is a feature toggle for soft-credit rollups; it has nothing to do with a payment card. A Protected Custom Setting could not hold any of them: they are per-record business data, and the two on <i>Settings__c</i> are admin-visible configuration that the Settings console edits. • At this site: Feature toggle for soft-credit rollups; matches on <i>*credit*</i>. • What the rule checks: PMD's ProtectSensitiveData (AppExchange rule set) flags a custom field whose API name matches a list of sensitive-looking substrings, and asks that such data be stored in Protected Custom Settings or Protected Custom Metadata instead. EVIDENCE Each field's <code><description></code> in <code>base/main/default/objects/</code> says what it holds. The package stores no secret in any Custom Setting, protected or not — the org-wide credentials live only in Named Credentials (FP-4 in docs/security/appexchange-security-review-readiness.md), which is what the rule's remediation is for. SOURCES • Code Analyzer's own rule definition (AppExchange_xml.xml inside the bundled sfca-pmd-rules jar, class <code>com.salesforce.security.pmd.xml.DetectSecretsInCustomObjects</code>): the rule message is "To store secrets, use Protected Custom settings or Protected Custom metadata." It is keyed on the field's API name, which is why an ordinary word that contains <i>*sign*</i>, <i>*auth*</i>, <i>*token*</i> or <i>*credit*</i> is reported with the message "is a potential auth token".

Reported finding	Location	Response
		• https://trailhead.salesforce.com/content/learn/modules/secure-secrets-storage/protect-secrets-using-platform-features — the Trailhead module the rule description refers to; it defines the remediation as storing "API keys or other secret keys" in protected custom metadata or protected custom settings. None of these fields holds a key. • https://developer.salesforce.com/docs/platform/salesforce-code-analyzer/guide/rules-pmd-appexchange.html#protectsensitivedata — the scanner's reference for this rule (page could not be retrieved while this document was prepared: the host answers automated requests with HTTP 403, so it is cited as the scanner's own reference link and not quoted). • base/main/default/objects/<Object>/fields/<Field>.field-meta.xml — each flagged field's <code><description></code> and <code><inlineHelpText></code> state what it holds.

LoadJavaScriptHtmlScript — 2 findings (pmd, High; AppExchange, Security, Visualforce)

Reported finding	Location	Response
R-022 LoadJavaScriptHtmlScript Working as designed	pages/stripePayment.page:53 Javascript should always be loaded from static resources	SCANNER'S CLAIM <i>Javascript should always be loaded from static resources</i> — reported by LoadJavaScriptHtmlScript (pmd, High) at pages/stripePayment.page:53 WHY THE CLAIM IS FALSE The claim that this script must be loaded from a static resource is false because Stripe forbids it: Stripe's integration documentation requires Stripe.js to be loaded directly from <code>https://js.stripe.com</code> and never bundled or self-hosted. The library is what keeps card data out of the merchant's origin, which is the basis of the package's PCI SAQ A eligibility, and Stripe patches it in place without a version change; a copy frozen into a static resource would be a payment library that is out of date the day the package is built and would void that eligibility. The page exists only to host the Stripe Payment Element in an iframe, the host <code>js.stripe.com</code> is declared as a packaged CSP trusted site, and the tag is preceded by a comment that states this reason. No Lightning Web Component in the package loads any external script. SUPPORTING DETAIL Stripe.js loaded from js.stripe.com in the two Visualforce payment pages. Stripe.js is the one script that cannot be bundled. Stripe requires it to be loaded from <code>https://js.stripe.com/v3/</code> at page load and forbids self-hosting, because the library is what keeps card data out of the merchant's origin (PCI DSS SAQ A eligibility) and Stripe patches it in place without a version change. Loading a frozen copy would ship a payment library that is out of date the day it is built. The package confines the load to exactly these two Visualforce pages, which exist only to host the Payment Element in an iframe and to receive the 3-D Secure / bank-authentication redirect; no Lightning Web Component loads any external JavaScript or CSS. • At this site: <code><script src="https://js.stripe.com/v3/"></code> on the iframe host page for the Payment Element. • What the rule checks: PMD's LoadJavaScriptHtmlScript (AppExchange rule set) flags a <code><script src=</code> that points at an external host. The review expects JavaScript to ship inside the package as a static resource, so that the code Salesforce reviewed is the code that runs. EVIDENCE <code>grep -rn 'loadScript\\ loadStyle' base/main/default/lwc</code> returns nothing, and these two pages are the only <code><script src=</code> in the package. <code>js.stripe.com</code> is a packaged CSP trusted site. The exception is written up as FP-9 in docs/security/appexchange-security-review-readiness.md, and the pages carry the reason in a comment beside the tag. SOURCES • https://docs.stripe.com/js/including — "Include the Stripe.js script on each page of your site—it should always be loaded directly from https://js.stripe.com, rather than included in a bundle or hosted yourself." • https://developer.salesforce.com/docs/platform/salesforce-code-analyzer/guide/rules-pmd-appexchange.html#loadjavascripthtmlscript — the scanner's reference for this rule (page could not be retrieved while this document was pre

Reported finding	Location	Response
		pared: the host answers automated requests with HTTP 403, so it is cited as the scanner's own reference link and not quoted). • base/main/default/cspTrustedSites/Stripe_js.cspTrustedSite-meta.xml — <code>js.stripe.com</code> is a packaged CSP trusted site, so the load is declared to the platform rather than slipped past it. • docs/security/appexchange-security-review-readiness.md, FP-9 — the standing exception and its PCI reasoning.

Reported finding	Location	Response
R-023 LoadJavaScriptHtmlScript Working as designed	pages/stripePaymentReturn.page:40 Javascript should always be loaded from static resources	SCANNER'S CLAIM <i>Javascript should always be loaded from static resources</i> — reported by LoadJavaScriptHtmlScript (pmd, High) at pages/stripePaymentReturn.page:40 WHY THE CLAIM IS FALSE The claim is false for the same reason: Stripe requires Stripe.js to be loaded from https://js.stripe.com and forbids self-hosting. This page is the return target of a 3-D Secure or bank-authentication redirect and must call Stripe.js to read the redirect result; it has no other content. The host is a packaged CSP trusted site, the reason is written in a comment beside the tag, and no other <code><script src></code> exists in the package. SUPPORTING DETAIL Stripe.js loaded from js.stripe.com in the two Visualforce payment pages. Stripe.js is the one script that cannot be bundled. Stripe requires it to be loaded from https://js.stripe.com/v3/ at page load and forbids self-hosting, because the library is what keeps card data out of the merchant's origin (PCI DSS SAQ A eligibility) and Stripe patches it in place without a version change. Loading a frozen copy would ship a payment library that is out of date the day it is built. The package confines the load to exactly these two Visualforce pages, which exist only to host the Payment Element in an iframe and to receive the 3-D Secure / bank-authentication redirect; no Lightning Web Component loads any external JavaScript or CSS. • At this site: Same tag on the 3-D Secure / bank-authentication return page, which must call Stripe.js to read the redirect result. • What the rule checks: PMD's LoadJavaScriptHtmlScript (AppExchange rule set) flags a <code><script src></code> that points at an external host. The review expects JavaScript to ship inside the package as a static resource, so that the code Salesforce reviewed is the code that runs. EVIDENCE <code>grep -rn 'loadScript\\ loadStyle' base/main/default/lwc</code> returns nothing, and these two pages are the only <code><script src=</code> in the package. js.stripe.com is a packaged CSP trusted site. The exception is written up as FP-9 in docs/security/appexchange-security-review-readiness.md, and the pages carry the reason in a comment beside the tag. SOURCES • https://docs.stripe.com/js/including — "Include the Stripe.js script on each page of your site—it should always be loaded directly from https://js.stripe.com, rather than included in a bundle or hosted yourself." • https://developer.salesforce.com/docs/platform/salesforce-code-analyzer/guide/rules-pmd-appexchange.html#loadjavascripthtmlscript — the scanner's reference for this rule (page could not be retrieved while this document was prepared: the host answers automated requests with HTTP 403, so it is cited as the scanner's own reference link and not quoted).

Reported finding	Location	Response
		- base/main/default/cspTrustedSites/Stripe_js.cspTrustedSite-meta.xml — <code>js.stripe.com</code> is a packaged CSP trusted site, so the load is declared to the platform rather than slipped past it. - docs/security/appexchange-security-review-readiness.md, FP-9 — the standing exception and its PCI reasoning.

Appendix A — every reported finding

Rule	Engine	Sev.	File:line	Message	Response	Verdict
ApexDangerousMethods	pmd	Moderate	base/main/default/classes/donation/domain/DonationDesignations.cls:36	Calling potentially dangerous method	R-001	Working as designed
ApexDangerousMethods	pmd	Moderate	base/main/default/classes/donor/domain/Contacts.cls:52	Calling potentially dangerous method	R-002	Working as designed
ApexDangerousMethods	pmd	Moderate	base/main/default/classes/recurring/domain/RecurringDonations.cls:61	Calling potentially dangerous method	R-003	Working as designed
ProtectSensitiveData	pmd	Moderate	base/main/default/objects/Campaign_Design__c/fields/Theme_Tokens_JSON__c.field-meta.xml:1	Theme_Tokens_JSON__c is a potential auth token in a custom object: Campaign_Design__c	R-004	False positive
ProtectSensitiveData	pmd	Moderate	base/main/default/objects/Donation_Designation__c/fields/Designation__c.field-meta.xml:1	Designation__c is a potential auth token in a custom object: Donation_Designation__c	R-005	False positive
ProtectSensitiveData	pmd	Moderate	base/main/default/objects/Donation_Staging__c/fields/Access-Token__c.field-meta.xml:1	Access-Token__c is a potential auth token in a custom object: Donation_Staging__c	R-006	Working as designed
ProtectSensitiveData	pmd	Moderate	base/main/default/objects/Donation_Staging__c/fields/Idempotency_Key__c.field-meta.xml:1	Idempotency_Key__c is a potential auth token in a custom object: Donation_Staging__c	R-007	Working as designed
ProtectSensitiveData	pmd	Moderate	base/main/default/objects/Guest_Request__c/fields/Idempotency_Key__c.field-meta.xml:1	Idempotency_Key__c is a potential auth token in a custom object: Guest_Request__c	R-008	Working as designed
ProtectSensitiveData	pmd	Moderate	base/main/default/objects/Payment_Account__c/fields/Publishable_Key__c.field-meta.xml:1	Publishable_Key__c is a potential auth token in a custom object: Payment_Account__c	R-009	Working as designed
ProtectSensitiveData	pmd	Moderate	base/main/default/objects/Payment_Account__c/fields/Stripe_Admin_Restricted_Key__c.field-meta.xml:1	Stripe_Admin_Restricted_Key__c is a potential auth token in a custom object: Payment_Account__c	R-010	Working as designed
ProtectSensitiveData	pmd	Moderate	base/main/default/objects/Payment_Account__c/fields/Stripe_Donor_Restricted_Key__c.field-meta.xml:1	Stripe_Donor_Restricted_Key__c is a potential auth token in a custom object: Payment_Account__c	R-011	Working as designed
ProtectSensitiveData	pmd	Moderate	base/main/default/objects/Portal_Access-Token__c/fields/Code_Hash__c.field-meta.xml:1	Code_Hash__c is a potential auth token in a custom object: Portal_Access-Token__c	R-012	Working as designed
ProtectSensitiveData	pmd	Moderate	base/main/default/objects/Portal_Access-Token__c/fields/Session_Hash__c.field-meta.xml:1	Session_Hash__c is a potential auth token in a custom object: Portal_Access-Token__c	R-013	Working as designed
ProtectSensitiveData	pmd	Moderate	base/main/default/objects/Recurring_Designation__c/fields/Designation__c.field-meta.xml:1	Designation__c is a potential auth token in a custom object: Recurring_Designation__c	R-014	False positive
ProtectSensitiveData	pmd	Moderate	base/main/default/objects/Recurring_Donation__c/fields/Designation__c.field-meta.xml:1	Designation__c is a potential auth token in a custom object: Recurring_Donation__c	R-015	False positive
ProtectSensitiveData	pmd	Moderate	base/main/default/objects/Recurring_Donation__c/fields/Re_Auth_Link_Sent_At__c.field-meta.xml:1	Re_Auth_Link_Sent_At__c is a potential auth token in a custom object: Recurring_Donation__c	R-016	False positive
ProtectSensitiveData	pmd	Moderate	base/main/default/objects/Recurring_Donation__c/fields/Re_Auth_Required__c.field-meta.xml:1	Re_Auth_Required__c is a potential auth token in a custom object: Recurring_Donation__c	R-017	False positive

Rule	Engine	Sev.	File:line	Message	Response	Verdict
ProtectSensitiveData	pmd	Moderate	base/main/default/objects/Settings__c/fields/Default_Designation__c.field-meta.xml:1	Default_Designation__c is a potential auth token in the object: Settings__c with public visibility	R-018	False positive
ProtectSensitiveData	pmd	Moderate	base/main/default/objects/Settings__c/fields/Email_Signature__c.field-meta.xml:1	Email_Signature__c is a potential auth token in the object: Settings__c with public visibility	R-019	False positive
ProtectSensitiveData	pmd	Moderate	base/main/default/objects/Settings__c/fields/Portal_Token_TTL_Hours__c.field-meta.xml:1	Portal_Token_TTL_Hours__c is a potential auth token in the object: Settings__c with public visibility	R-020	False positive
ProtectSensitiveData	pmd	Moderate	base/main/default/objects/Settings__c/fields/Soft_Credit_Automation_Enabled__c.field-meta.xml:1	Soft_Credit_Automation_Enabled__c is a potential auth token in the object: Settings__c with public visibility	R-021	False positive
LoadJavaScriptHtmlScript	pmd	High	base/main/default/pages/stripePayment.page:53	Javascript should always be loaded from static resources	R-022	Working as designed
LoadJavaScriptHtmlScript	pmd	High	base/main/default/pages/stripePaymentReturn.page:40	Javascript should always be loaded from static resources	R-023	Working as designed